

Registry, Compiler, and Cloud Memory: Toward a New Generation of Agent Architecture Centered on an Agent Operating System

Abstract

Autonomous agents based on large language models (LLMs) are rapidly evolving from conversational interactions to complex task execution. However, their memory and communication mechanisms remain trapped in the primitive paradigm of "context as memory": system instructions, role settings, historical dialogues, retrieved fRAGments, and tool outputs are indiscriminately mixed into a linear context window. This leads to ambiguous state management, redundant computational resources, frequent context pollution, and a dual dilemma of data privacy and commercial sustainability. The industry has long relied on application-layer optimizations such as prompt engineering, retrieval-augmented generation (RAG), memory summarization, and context expansion, yet has failed to break through the underlying logic of "full data transfer."

This paper moves beyond application-layer patching and systematically transfers the principles of classic operating systems—layered abstraction, structured state management, compiled communication, and resource isolation—into the agent design space, proposing a complete AgentOS architecture. This architecture establishes a three-tier hierarchical system: "local memory layer - compiled communication layer - cloud persistence layer." Its core innovations consist of three pillars: (1) A four-table separated local memory system: Agent memory is decomposed into four logically independent tables—Registry, Environment Variables, System Logs, and Memory Store—which respectively carry deterministic metadata, runtime parameters, behavioral audit records, and raw unstructured data, enabling constant-time deterministic queries and physical data isolation. (2) The Compiled Work Order communication protocol: An independent Compiler serves as an intent purification and instruction translation role, completely separating human-machine natural language interaction from machine-machine structured instruction communication. Through standardized work orders, minimum necessary information

transmission is achieved, addressing context pollution and redundant transmission at the architectural level. (3) The Cloud Memory Pointer Service: This overturns the inefficient "cloud stoRAGe - local transfer - re-upload" pipeline by directly retaining cognitive assets in dedicated memory space co-located with the inference engine. Through an explicit cache handshake protocol, programmable persistent context is realized, enabling co-located coordination of cognitive assets and computational resources.

This paper rigorously defines the AgentOS operating system primitives, process management model, interrupt handling mechanism, distributed CAP trade-offs, and complete threat model. It provides full engineering details, from system call interfaces, Registry Schema definitions, and the complete Work Order Protocol specification to a reference Compiler implementation. A standardized evaluation framework, including ablation experiments, is designed to verify the independent contributions of each component. AgentOS not only improves memory access efficiency and state determinism by orders of magnitude but also introduces an architectural privacy philosophy of "configuration in the cloud, materials on the edge, pointers isolated," opening a new commercial paradigm from "pay-per-token" to "subscribe-by-intelligence-density." This paper argues that AgentOS is not a single technical optimization but a paradigm revolution that lays the infrastructure for building the Agent Internet, with the potential to become the industry standard for the next generation of intelligent agents.

Keywords: Agent Architecture; Agent Operating System; State Management; Context Optimization; Hierarchical Memory; Compiled Communication; Cloud Memory; Privacy Protection; Multi-Agent Collaboration

1. Introduction

1.1 "The Warehouse and the Truck": A Paradox That Must Be Broken

Imagine a warehouse manager. His warehouse holds millions of items, neatly categorized and organized. Yet, whenever a customer order arrives—even for a single item—he must drive a giant truck, load all relevant goods from the warehouse, and deliver them to the customer, who then searches through the pile for the desired product. For every new order, this full-load process is repeated.

This is the true portrayal of the vast majority of current AI Agent memory systems.

The "warehouse" is a meticulously designed memory system: unstructured long-text abstracts, vector retrieval databases, and knowledge graphs. The "truck" is the context window of the large model. The "order" is each user request. The industry invests immense effort in optimizing the warehouse: more precise semantic retrieval, more reasonable memory decay curves, and more refined entity-relationship extraction. These efforts are highly valuable in themselves, but they collectively fall into a blind spot — never questioning the act of "having to load all goods onto the

truck."

The more refined the memory, the more "goods" need to be injected into the context. The more bloated the context, the greater the computational cost and the higher the expense. The response from large model vendors is to provide increasingly larger context windows — from 4K to 128K to 1M Tokens — which is akin to giving the administrator a bigger truck without changing the fundamental nature of full-scale transportation. Memory grows infinitely, but the window has an upper limit. Moreover, the longer the context, the higher the inference cost and latency, and the model's recall accuracy for key information within a long context actually decreases. This is a path of a self-defeating approach, which we term the "Memory Wall" dilemma for agents.

1.2 The Lack of System-Level Thinking

The root of this dilemma lies in treating the Agent merely as an "application." At the application layer, we use techniques like prompt engineering, retrieval-augmented generation (RAG), and memory abstracts to simulate the appearance of human memory, but we have never examined the Agent's software stack from the perspective of an "operating system."

Looking back at the history of personal computing. In the DOS era, users had to completely reconstruct the computer's current state in their minds with every command: the current directory, available files, the result of the last operation. The system had no unified state management; all states relied on user memory. How similar this is to today's Agents "recalling" their own state by reading lengthy contexts! The revolutionary breakthrough of modern operating systems lies in providing abstraction and isolation: the file system manages persistence (users don't need to know the physical sectors of files), the Registry stores system and application configuration metadata (modifying configurations doesn't require changing code), the process manager maintains runtime state and resource allocation (each process has its own independent, protected memory space), and system logs independently record behavioral audits. These components have clear responsibilities and communicate through efficient system calls, rather than reading each other's "lengthy texts." When you click an icon in Windows 11, the system doesn't read a 100,000-word "System User Manual"; it only queries a lightweight key-value pair in the Registry to complete the entire state loading.

What the Agent lacks is precisely this layer of "system."

1.3 Core Proposition and Contributions of This Paper

This paper proposes a core proposition: a memory and communication infrastructure isomorphic to classic operating systems should be built for autonomous agents, namely AgentOS. This is not a rough analogy, but a complete set of architectural principles that can be strictly defined, engineered for implementation, and verified through evaluation.

The core contributions of this paper are as follows:

1. **Layered Reconstruction of Local Memory:** Proposes a "Registry - Environment Variables - System Logs - Memory Store" four-table separation model, completely decoupling deterministic metadata, runtime parameters, behavioral audit records, and raw unstructured data, achieving constant-time deterministic reading of state. Among these, the Registry is the core innovation — unlike knowledge graphs (which attempt to describe the relationships of everything), it only records the configuration and state of "me" and "my environment," enabling atomic read/write operations through a tree-structured hierarchical key-value store.
2. **Compilation Paradigm Shift in Communication:** Proposes a "Compiler + Work Order Protocol" two-layer communication architecture, separating natural language interaction from structured inter-machine communication, achieving intent purification, instruction standardization, and minimum necessary information transmission, addressing context pollution and redundant transmission at the architectural level. The Compiler adopts a two-level implementation of L1 rules + L2 models, with zero model invocation cost for high-frequency, simple operations.
3. **Cloud Memory Revolution:** Proposes the Cloud Memory Pointer Service, overturning the inefficient "cloud stoRAGe - local transfer - re-upload" pipeline, enabling cognitive assets to reside co-located with the inference engine. Through an explicit cache handshake protocol, it achieves programmable persistent context, defining a three-tier progressive implementation path: L1 software-defined → L2 near-inference memory pool → L3 co-located fusion.
4. **Architectural Privacy and Security Model:** Through the three principles of "separation of configuration and material, separation of pointers and data, and separation of companionship and execution," it achieves native privacy protection at the architectural level without relying on complex encryption algorithms — only pointers and configurations are stored in the cloud, while raw data always remains local.
5. **Complete System Abstraction and Evaluation Framework:** Supplements core operating system primitives such as process management, system calls, and interrupt handling, clarifies the distributed CAP trade-offs and threat model, and designs a standardized evaluation framework including ablation experiments.
6. **Novel Business and Ecosystem Paradigm:** Demonstrates how AgentOS can drive the industry from "pay-per-Token" to "subscription based on intelligence density," building a strong-lock commercial moat centered on Cloud Memory cognitive assets, and laying the Work Order Protocol foundation for a multi-agent collaborative internet.

2. Related Work

Research on Agent memory and communication architectures spans multiple fields, including natural language processing, multi-agent systems, operating systems, and distributed systems. This chapter reviews existing approaches along four technical lines and precisely identifies the innovation boundary of AgentOS.

2.1 Long Context and Virtual Memory Approaches

The "LLM as OS" approach, represented by MemGPT [2], simulates the virtual memory mechanism of an operating system through paging scheduling between the main context and external stoRAGe, breaking through the limitation of a fixed context length. Its core mechanism divides the conversation history into two layers: "main context" and "external stoRAGe." When information exceeds the main context capacity, the model autonomously calls functions to swap out expired content to external stoRAGe, and retrieves it back when needed. However, the essence of this approach is still time-division multiplexing of a single large context: all information (system instructions, memory abstracts, retrieved fRAGments) is still mixed within the same context in natural language form, and state switching relies entirely on the model's own semantic interpretation of the abstract text, rather than deterministic structured queries. Its core optimization point is the expansion of context length, not the reconstruction of the memory architecture. Furthermore, the model's autonomous scheduling of memory swapping itself consumes Tokens and inference resources, and the scheduling overhead is non-negligible in long tasks.

AgentOS takes a critical step forward by completely stripping deterministic state information from the context, constructing an independent Registry layer, and implementing structured read/write operations through a Compiler. This eliminates the uncertainty of semantic interpretation entirely, achieving a leap from "virtual memory simulation" to a "complete operating system architecture." Registry lookups in AgentOS are constant-time key-value addressing, consuming no model inference resources.

2.2 Structured Memory Schemes

Knowledge graph memory (e.g., GraphRAG [24]) and entity memory attempt to extract dialogue content into triples (subject, relation, object) or structured attribute pairs, storing them separately to improve query efficiency and reasoning capabilities. GraphRAG preprocesses documents into entity-relation networks and performs graph traversal retrieval during inference. Such schemes attempt to forcibly structure all information, leading to extraction costs that rise exponentially with dialogue scale — in open-domain scenarios, the error rates for named entity recognition and relation extraction are already high, and errors accumulate and propagate in subsequent extractions. Furthermore, triple structures struggle to capture ambiguity, irony, and implicit semantics in natural language, falling into the trap of "over-engineering."

AgentOS adopts a fundamentally different strategy: it structures only deterministic metadata (configurations, states, pointers), while retaining raw unstructured data in the memory store for on-demand retrieval. The Registry is not a miniature knowledge graph; it does not describe "the relationship between the user and

Project X," but rather records its own configuration, such as ``HKEY_CURRENT_USER\ProjectRegistry\Proj_X\Status = "Active"`. This achieves the efficiency of structured queries (constant-time key-value addressing) while avoiding the engineering disaster of full-scale structuring.`

2.3 Multi-Agent Communication Protocol

The traditional FIPA-ACL protocol [23] for multi-agent systems defines complex semantic message formats, including fields such as sender, receiver, content language, and ontology, supporting various speech acts like request, inform, and promise. However, practical deployment is difficult because it relies on the receiver to perform complex semantic parsing of the message, and the protocol itself is overly heavy — the metadata overhead of each message can exceed the actual content. In recent years, collaboration between large model agents has often used natural language dialogue for communication. While this offers good flexibility, it suffers from low information density, high ambiguity, and poor communication efficiency.

AgentOS's Work Order Protocol pioneers a third path: based on the Compiler's pre-processing mechanism, the user's natural language is first transformed by the Compiler into structured instructions, which are then sent to the execution model or other agents. This preserves the flexibility of natural language as a human-machine interface, but replaces it with a structured protocol for machine-machine communication, significantly reducing communication overhead and ambiguity. The ``instruction.op`` field of the protocol directly declares the operation type, allowing the receiver to execute it without needing to "understand" the intent.

2.4 Context Caching Technology

Since 2024, vendors such as Google and Anthropic have successively introduced prompt caching features [10, 12]. Google Vertex AI's Context Caching allows users to specify content to be cached, with the server reusing its KV tensors within the TTL; Anthropic's Prompt Caching automatically identifies and caches repeated prefixes. This technological direction confirms our insight into "wasteful repeated computation," but existing solutions have fundamental limitations: caching strategies (eviction timing, time-to-live, scope of effect) are controlled by the vendor as a black box, preventing developers from using it as a reliable persistent memory layer — you never know when the cache will be evicted; the cached content consists of raw prompt blocks, not structured, directly injectable cognitive assets; and the APIs from different vendors are incompatible, preventing the formation of a cross-platform standard.

AgentOS elevates this capability into an explicit, programmable Cloud Memory protocol. Developers autonomously manage the full lifecycle (creation, reference, update, invalidation) of cognitive assets through standardized instructions ``cache_store`/`cache_load``. The server provides clear commitments on the cache's lifecycle (TTL guarantees), and the cached content consists of compiled, structured cognitive assets. This achieves an upgrade from "passive optimization" to "active architecture."

2.5 Horizontal Comparison of Existing Solutions

Solution Type	Representative Product	Core Paradigm	State Determinacy	Context Efficiency	Privacy Protection	Ecosystem Openness
Pure Long Context	GPT-4 128K	Context as Memory	Extremely low, entirely dependent on semantic interpretation	Extremely low, full retransmission	Poor, all raw data uploaded	High, general-purpose interface
Application-layer Framework	LangChain	Context + RAG Enhancement	Low, dependent on memory Abstract	Low, retrieval segment injection	Poor, requires uploading segments	High, open-source and general-purpose
Virtual Memory Solution	MemGPT	Context Paging Scheduling	Medium, dependent on model scheduling	Medium, page swapping in and out	Medium, data stored locally	Medium, specific architecture
Managed Agent	Assistants API	Server-side Context Management	Medium (black-box)	Medium, server-side reuse	Poor, data retained by the vendor	Low, vendor lock-in
Structured	GraphRAG	Triple-based knowledge	Relatively high, graph	Medium, structured result	Medium, requires upload	Medium, specific scenario

memory		e graph	query	injection	and extraction	s
AgentOS	-	Operating system layered architecture	Extremely high, structured key-value query	Extremely high, only transmits work orders and increments	Extremely high, architecture-level isolation	High, open protocols

3. Core Design Philosophy of AgentOS

3.1 Architecture Over Algorithm

This is the first principle of AgentOS. Existing solutions tend to use complex algorithms to fix a fundamentally flawed architecture — using more sophisticated retrieval ranking and more complex memory weight adjustments to compensate for the structural inefficiency of "full-context injection." However, architecture determines the responsibilities and boundaries of system components: a flawed architecture, no matter how ingenious its internal algorithms, cannot break through its essential bottlenecks; a correct architecture, on the other hand, can achieve outstanding performance with extremely simple algorithms. The paged virtual memory mechanism of classic operating systems, based on simple page table mapping and an LRU replacement strategy, is far more efficient than any complex memory allocation algorithm designed in real mode.

The design focus of AgentOS has always been on defining components and their boundary relationships, rather than on specific memory scoring formulas. Using the simplest structured key-value query (constant-time complexity) can achieve higher state certainty than the most complex semantic retrieval — this is the power of architecture.

3.2 Layering Over Mixing

"Mixing" is the root of all evil in current architectures. System instructions, role settings, historical dialogue, retrieved snippets, and tool outputs are all mixed as undifferentiated token sequences within the same context window, causing unclear responsibilities, severe contamination, and difficulty in debugging. When an agent behaves abnormally, you cannot quickly locate whether the system prompt was contaminated, a wrong snippet was retrieved, or a piece of casual conversation in the history interfered with the judgment. AgentOS strictly adheres to the layering principle, decomposing the agent's complete capability stack into a local memory layer, a

compilation communication layer, and a cloud persistence layer. Each layer is further subdivided into components, which interact through clearly defined system call interfaces. This separation allows each component to evolve and optimize independently, and to quickly locate the root cause when problems arise.

3.3 Pointers Over Data

This is the core idea running through all data design in AgentOS. We do not move data; we move references to data. In a traditional Agent Architecture, learning that "the user prefers a concise style" requires injecting the full descriptive text into the context; in AgentOS, this description is stored in the memory store, and only a pointer, ``user_pref_style_id = 0x4A``, is passed at runtime. Registry key-values, environment variables, and Cloud Memory cache IDs are essentially pointers at different levels. They are extremely small (typically strings or numeric IDs), extremely fast, and natively support management operations such as version control, access control, and invalidation handling. More importantly, pointers naturally achieve physical separation between the reference and the data — even if a pointer is leaked, an attacker cannot obtain the original data through it.

3.4 Protocols Over Prompts

Prompts are designed for humans; they are ambiguous and context-dependent. When the interaction shifts from "human-model" to "system-model," using prompts becomes inefficient and unreliable — the model may "misunderstand" the prompt's intent, prompts may conflict with each other, and crafting prompts is itself an "art." AgentOS advocates for the use of structured protocols in internal system communication. A Compiler can translate a user's vague instruction, "Help me continue the last analysis," into a precise protocol command `{op: "RESUME_TASK", task_id: 0x334, params: {...}}`. Upon receiving this, the model does not need to "understand" what "continue" means; it simply executes the corresponding operation according to the protocol specification. This is analogous to a CPU executing machine code rather than reading an English manual — communication between machines is always more efficient and deterministic with structured protocols.

4. Local Memory System Design: Four-Table Separation and OS Abstraction

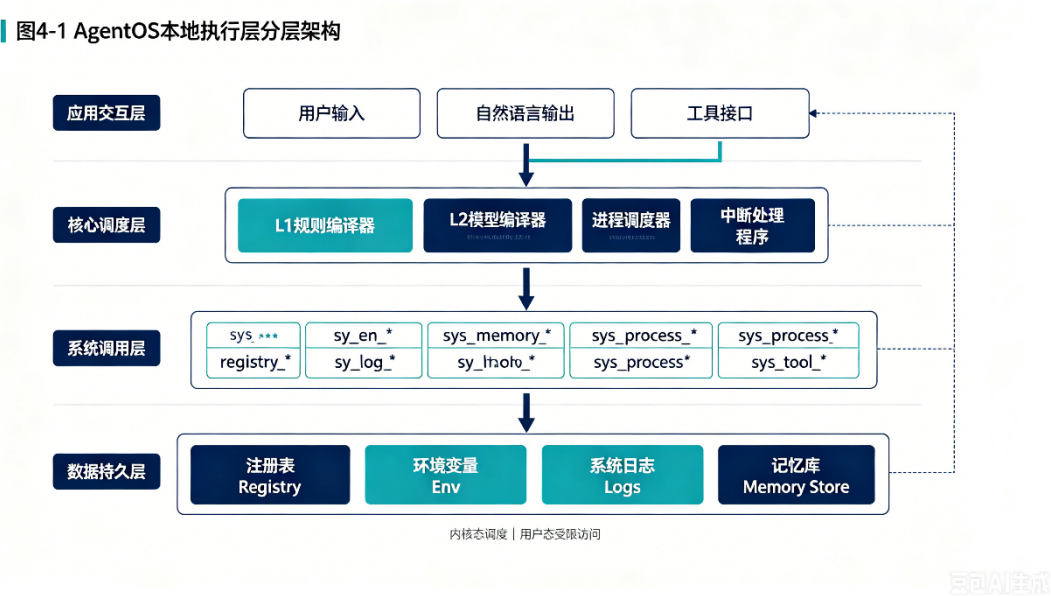
The local memory layer serves as the user-space runtime environment of AgentOS and is the single source of truth for the entire system's state. This chapter elaborates on the four-table separation memory model, operating system primitives and system call interfaces, the process management model, and the interrupt handling mechanism.

4.1 Overall System Architecture

The local environment of AgentOS adopts a layered design, from bottom to top as follows:

1. **Data Persistence Layer: A four-table separated stoRAGe system, including Registry, Environment Variables, System Logs, and Memory Store.**
2. **System Call Layer: A standardized kernel call interface that provides controlled access to underlying resources.**
3. **Core Scheduling Layer: The Compiler acts as the scheduler, responsible for process management, interrupt handling, and instruction dispatching.**
4. **Application Interaction Layer: The interface for interaction with users and external tools.**

Figure 4-1 Layered Architecture of the AgentOS Local Execution Layer



Description: Displays the hierarchical relationships and core components of the Data Persistence Layer, System Call Layer, Core Scheduling Layer, and Application Interaction Layer from bottom to top.

4.2 Four-Table Separation Memory Model

Based on data attributes, read/write patterns, and usage, all of the Agent's memory is decomposed into four logically independent tables:

Dimension	Registry	Environment Variables	System Logs	Memory Store
Core responsibilities	Configuration, state, meta-pointer	Runtime parameters	Behavioral audit trail	Raw data "data lake"

Data structure	Tree-structured hierarchical key-value pairs	Flat key-value pairs	Append-only time series	Unstructured text / vectors
Read-write pattern	Frequent reads, low-frequency atomic writes	Frequent reads, occasional writes	Append-only writes, offline reads	Low-frequency reads, low-frequency writes
Change frequency	Minute / task level	Second / session level	Real-time appending	Day / week level
Whether involved in inference	Yes, directly injected by the Compiler	Yes, injected by the Compiler	No, offline audit only	Passive scheduling by pointer
Determinism	Extremely high, atomic updates	Extremely high	Extremely high, immutable	Low, semantically ambiguous
Typical size	KB level	Byte level	MB level (rotatable)	GB–TB level
Classic OS counterpart	Control registers / Process Control Block	User-mode stack context	Kernel audit log	Disk file system

4.2.1 Registry: Deterministic Metadata Database

The Registry is the most core innovative component of AgentOS. Its design is inspired by the Windows Registry but has been comprehensively restructured for the state management needs of Agents. Its core responsibility is to store all deterministic, structured, and quickly indexable configuration and state information for the Agent and user. It does not store data itself, but only "data about data" and the "current running state."

The construction of the Registry does not require complex information extraction; instead, it is written deterministically directly by the Compiler during task execution. For example, when an Agent completes a data cleaning step, the Compiler does not generate a natural language statement like "I have completed the data

cleaning..." but atomically writes HKEY_CURRENT_SESSION\CurrentTask\Phase = "data_cleaning_complete".

Technical Note: The Latency of a Tree Path Query Is Independent of the Overall Registry Size and Depends Only On the Path Depth (typically ≤ 5 Levels). In Engineering Practice, It Can Be Regarded as a Constant-time Deterministic Operation, Rather Than a Strictly Theoretical $O(1)$ Complexity.

Registry Schema Definition:

Plaintext

4.2.2 Environment Variables: Runtime Instant Parameters

Environment variables store engineering parameters that need to be frequently read at runtime and may need to switch quickly between different sessions, decoupling them from the Agent's core memory.

Plaintext

Core Features:

- **Session-level isolation:** Different sessions have independent environment variables; switching sessions changes the parameters.
- **Immediate effect:** Modifying environment variables does not require changes to the Registry or prompts; model switching and parameter adjustments take effect immediately.
- **Sensitive information handling:** API keys and similar data are stored only in environment variables, not in the Registry or memory store, preventing persistent leakage.

4.2.3 System Log: Immutable Behavior Record

The system log is an append-only, immutable, timestamp-ordered recording system.

Plaintext

Standard event types:

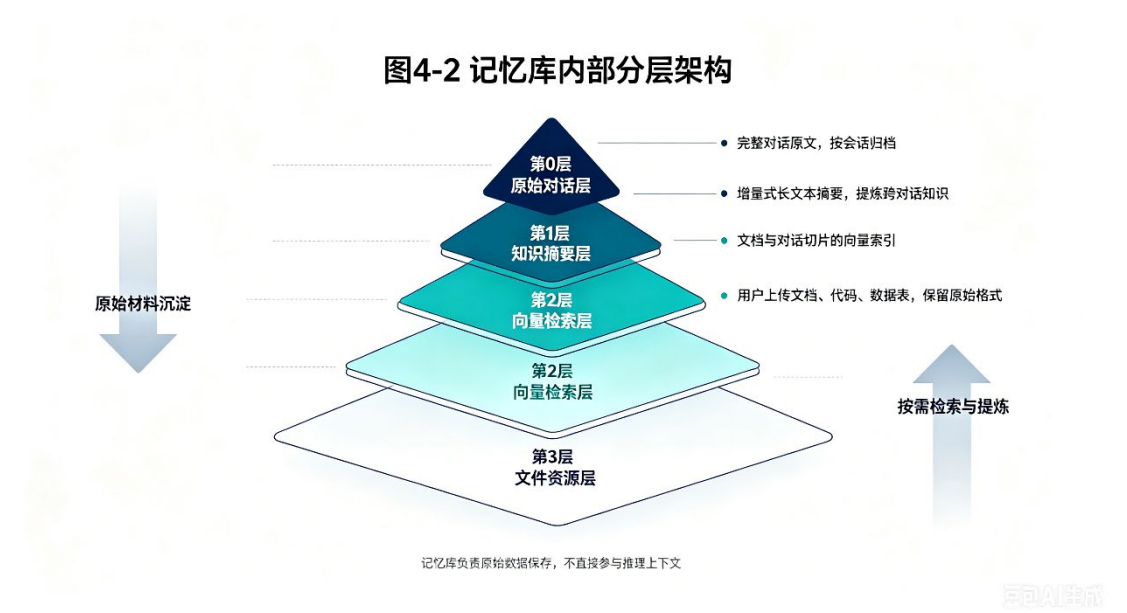
Event Category	Event Type	Trigger Scenario
Task Class	TASK.START, TASK.PAUSE, TASK.RESUME, TASK.COMPLETE,	Task Lifecycle Stages

	TASK.ERROR	
Registry Class	REGISTRY.SET, REGISTRY.DELETE, REGISTRY.CONFLICT	Registry Key-Value Changes
Tool Class	MCP.INSTALL, MCP.INVOKE, MCP.FAULT	MCP Tool Installation, Invocation, and Failures
Interaction Class	USER.INPUT, USER.INTERRUPT, AGENT.RESPONSE	User Interaction Events
Cloud Class	CLOUD.SYNC, CLOUD.LOAD, CLOUD.STORE	Cloud Synchronization and Cache Operations

4.2.4 Memory Store: Unstructured Data Lake

The memory store is the ultimate repository for all raw, unstructured, and semi-structured data. Its internal structure is layered as follows:

Figure 4-2 Internal Layered Architecture of the Memory Store



Description: Illustrates the four-layer structure of the memory store: raw text layer, vector index layer, semantic abstract layer, and pointer mapping layer.

4.3 Operating System Primitives and System Call Interface

To achieve strict layered isolation, AgentOS defines a standard system call interface.

The Compiler, acting as a kernel-mode scheduler, accesses underlying resources through system calls; upper-layer tasks and external agents can only interact with the system via system calls and cannot directly manipulate underlying data.

Permission model:

- **Kernel mode:** The Compiler and core system processes can execute all system calls.
- **User mode:** Business tasks, external agents, and third-party plugins can only execute restricted read-only calls and authorized operations.

Complete definition of the system call interface:

Plaintext

4.4 Process Management Model

AgentOS abstracts individual user tasks as Agent processes, drawing on the process management concepts of classic operating systems to achieve full lifecycle management of tasks.

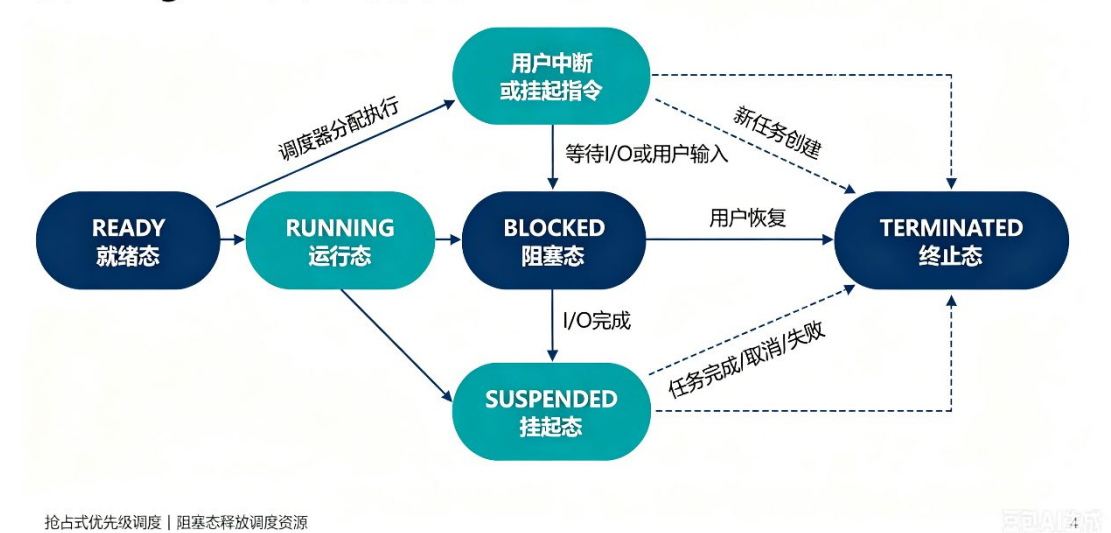
Process Control Block (PCB) structure:

Plaintext

Process state machine:

Figure 4-3 Agent Process State Transition Diagram

图4-3 Agent进程状态转换图



Description: Illustrates the transition conditions and triggering events among the five states: READY, RUNNING, BLOCKED, SUSPENDED, and TERMINATED.

Scheduling Policy: The Compiler acts as the scheduler, employing priority-based preemptive scheduling. High-priority tasks (e.g., user instant commands)

can preempt low-priority background tasks; tasks of the same priority are executed on a first-come, first-served basis; blocked tasks release scheduling resources and re-enter the ready queue upon event triggering.

4.5 Interrupt and Exception Handling

AgentOS establishes a complete interrupt handling mechanism, where all exception events are defined as standard interrupts and are managed by a unified interrupt handler, rather than relying on the model to handle them independently.

Interrupt Vector Table:

Interrupt Code	Interrupt Name	Trigger Scenario	Priority
INT_USER_OVERRIDE	User Interrupt	User interrupts mid-task, modifies instructions, switches tasks	Highest
INT_TOOL_FAULT	Tool Failure Interrupt	MCP call timeout, execution failure, malformed return	Medium-High
INT_RESPONSE_ERROR	Response Anomaly Interrupt	Model response does not conform to the work order schema, content violation	Medium
INT_CONCURRENCY_CONFLICT	Concurrency Conflict Interrupt	Registry concurrent write conflicts, cloud sync state inconsistency	Low

Interrupt Handler Pseudocode:

5. Compilation Communication Layer: Work Order Protocol and Compiler Implementation

If the local memory layer endows the Agent with structured self-awareness, then the compilation communication layer defines the language through which the Agent interacts with the world. This chapter elaborates on the two-tier Compiler architecture, the complete Work Order Protocol specification, the cognitive asset cache handshake protocol, and the complementary mechanism with RAG.

5.1 Compiler Core Positioning and Two-Tier Architecture

The Compiler is the core gateway and translator of AgentOS, connecting users, local systems, and cloud models. Its five core functions are: intent purification (stripping ambiguity and redundancy from natural language), state loading (querying the Registry and environment variables), work order generation (packaging into standardized work orders), result parsing (receiving and parsing structured responses), and response translation (converting structured results into natural language).

The Compiler is essentially a translator between human-machine language and machine-machine language, and also the first firewall against context pollution. All information entering or leaving the system must pass through the Compiler's filtering and conversion.

Two-Tier Compiler Implementation Scheme:

Figure 5-1 Two-Tier Compiler Architecture



Description: Illustrates the pipeline of the L1 rule-based Compiler and the L2 large model Compiler, including the complete process of intent input, rule matching, fallback on failure, and work order generation.

Complete Reference Implementation of the L1 Rule-Based Compiler:

Python

5.2 Complete Work Order Protocol Specification

A Work Order is the sole communication unit for interaction between an Agent and a cloud model, or between Agents. A work order is a structured protocol file, not natural language text.

Complete Work Order Protocol Schema (JSON Schema v1.0):

JSON

Complete Response Message Specification:

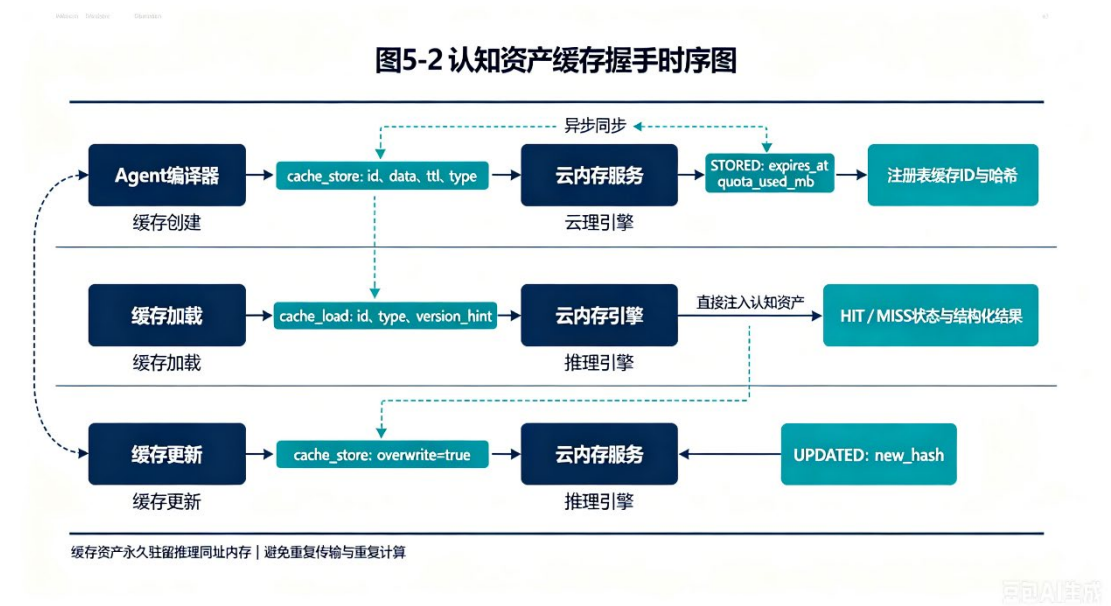
JSON

5.3 Cognitive Asset Cache Handshake Protocol

`cache\_load` and `cache\_store` together constitute the cognitive asset caching protocol of AgentOS, enabling explicit and programmable control over cloud-side context.

Complete cache handshake process:

Figure 5-2 Cognitive Asset Cache Handshake Sequence Diagram



Description: Shows the sequential interactions among the Compiler, Cloud Memory service, and inference engine, including ``cache_store``, ``cache_load``, asset injection, and result return.

5.4 Complementary Mechanism with RAG

AgentOS does not eliminate RAG; rather, it downgrades RAG from a "core memory backbone" to a "retrieval component of the memory store."

Plaintext

Complete invocation chain:

1. During task execution, the Compiler determines that historical knowledge needs to be invoked.
2. The Compiler sends a retrieval request to the memory store via ``sys_memory_search``.
3. The RAG engine within the memory store performs vector retrieval and returns the Top-K relevant segments.
4. The Compiler refines the retrieval results, extracting key conclusions and structured information.
5. The refined information is updated into the corresponding fields of the Registry or integrated into the ``new_delta`` of the work order.
6. The original retrieved segments are released and do not enter the final inference context.

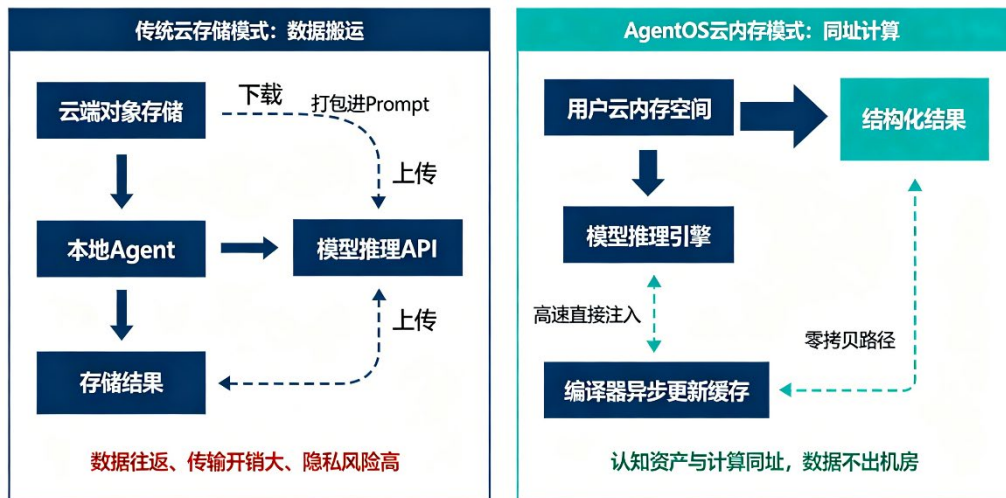
This approach preserves RAG's powerful semantic retrieval capability while avoiding the contamination and bloat caused by directly injecting retrieved content into the context.

6. Cloud Memory Architecture

6.1 Paradigm Shift: From Cloud StoRAge to Cloud Memory

The current model is the "cloud stoRAge" model: Agent skills and memories are stored in cloud object stoRAge (e.g., S3), downloaded locally at startup, and packaged into the Prompt for upload during execution. Data travels back and forth between local and cloud. AgentOS's "Cloud Memory" model completely restructures this pipeline: cognitive assets are stored directly in dedicated memory space co-located with the inference engine, and during inference, they are read directly from adjacent memory partitions and injected into the context.

图6-1 从云存储到云内存



Comparison of the two models:

Plaintext

6.2 Three-Layer Implementation Path

Tier	Name	Technical Implementation	Typical Latency	Applicable Phase
L1	Software-Defined Cloud Memory	Based on existing KV cache API encapsulation, SSD persistence + in-memory hot data	10-100 milliseconds	Immediate deployment
L2	Near-Inference Memory Pool	RDMA high-speed memory pool, resident in memory, bypassing business gateway	1-10 microseconds	Scale users

L3	Co-located Memory Fusion	GPU HBM memory partition persistence, MMU direct addressing, zero copy	10-100 nanoseconds	Ultimate form
----	--------------------------------	--	-----------------------	---------------

6.3 CAP Trade-offs and Consistency Model

- **Local Registry: Strong consistency (CP).** It serves as the single source of truth for state, and any update must be atomically written locally.
- **Cloud Memory: Eventual consistency (AP).** It acts as an asynchronous mirror and performance acceleration layer, with local state always taking precedence in case of conflicts.

Synchronization Mechanism:

Plaintext

7. Security and Privacy Model

7.1 Threat Model Definition

Attacker capability assumptions: The attacker may steal cloud-stored data (database breach attack), eavesdrop on public network transmission links (man-in-the-middle attack), obtain user work order data through social engineering or vulnerabilities, or attempt to tamper with the Compiler or inject malicious instructions.

The attacker is not assumed to be able to: physically breach the user's local terminal and obtain the highest privileges, induce the user to actively disclose sensitive data, or break the underlying encryption algorithm itself.

7.2 Three Principles of Separation

Principle One: Separation of Configuration and Material. The cloud stores only compiled configurations and skills (How), not the original private material (What). Even if Cloud Memory is breached, the attacker can only see meta-information such as "the user habitually writes detailed comments in Python."

Principle Two: Separation of Pointers and Data. The Registry stores pointers such as file paths, variable names, and hash values, rather than the data itself.

If a hacker obtains the Registry, they can only get path strings like `C:\...\Secret Plan.docx`, but cannot penetrate the user's local system to access the actual files—just like having the index card cabinet of a library, there remains a physical-world gap before they can actually read the book's content.

Principle Three: Separation of Interaction and Execution. The Compiler routes emotional companionship requests and task execution requests to different models. The execution model never comes into contact with natural language emotional content, preventing emotional contamination at the source.

7.3 Risk Mitigation Solutions

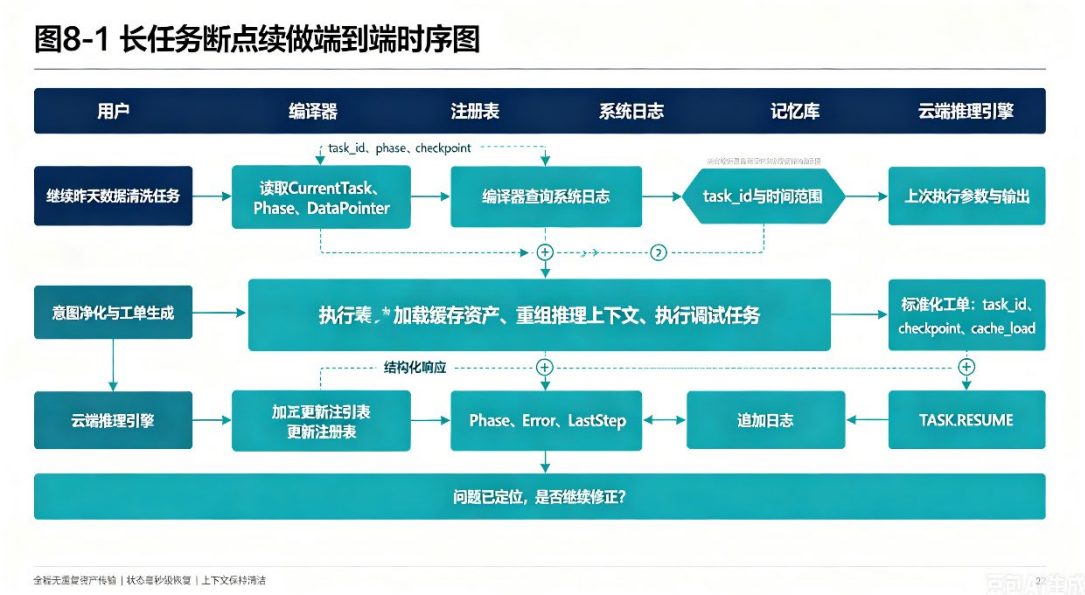
Risk	Description	Mitigation Strategy
Pointer side channel	Pointer access patterns reveal behavioral regularities	Access perturbation (insertion of random noise), batch aggregation requests, traffic padding
Compiler poisoning	Compiler tampered with, forged work orders	Hash verification (Registry key-value pairs appended with SHA256), operation logging and auditing, user confirmation required for core configuration changes
Temporary data leakage	Materials temporarily injected into sandbox during task execution	Sandbox isolation (independent sandbox per task), ephemeral disposal (memory cleared upon completion), differential privacy noise (applicable only to statistical analysis tasks; disabled for precise computation scenarios such as code generation)

8. End-to-End Workflow

Taking "Continue yesterday's data cleaning task" as an example, the complete

sequential flow is shown:

Figure 8-1 End-to-End Sequence Diagram for Long-Task Breakpoint Resumption



Description: Using "Continue yesterday's data cleaning task" as an example, it illustrates the complete sequential flow of user input, Compiler state loading, Cloud Memory asset injection, inference execution, and result return.

No duplicate asset transmission occurs throughout the process, state recovery is instantaneous (millisecond-level), and the context remains clean at all times.

9. Experimental Evaluation Design

9.1 Baseline Selection

Baseline Number	Scheme Type	Specific Implementation
Baseline 1	Pure long-context	128K context window, full conversation history directly concatenated
Baseline 2	RAG enhancement	Vector database stores conversation segments, Top-K=5 retrieval injected into context

Baseline 3	Virtual context (MemGPT style)	Paged memory management, model autonomously schedules memory swapping in and out
------------	--------------------------------	--

9.2 Datasets and Metrics

We adopt the AgentBench standard benchmark [21] combined with a custom 30-round long-task sequence (covering multi-step data processing, code development and debugging, task switching and interruption recovery, interspersed with 5 rounds of casual conversation interference).

Metric Category	Specific Metric	Measurement Method
Efficiency	AveRAGe input tokens per turn, token growth slope, P95 end-to-end latency	Count input tokens per API call per turn, compute slope via linear regression
Accuracy	State recovery accuracy, deterministic information recall F1 score, long-task completion rate	Compare recovered state with ground truth, compute exact match rate
Robustness	Anti-contamination decay rate, exception handling success rate	The percentage decrease in task completion rate after interference insertion relative to the interference-free baseline
Cost	Total token consumption, per-task cost	Total input and output tokens aggregated over the complete 30-round task sequence

9.3 Ablation Experiments Design

Ablation Version	Removed	Implementation	Verification
------------------	---------	----------------	--------------

	Component	Method	Objective
A1	Remove Registry	State information is instead carried through context	Verify the contribution of Registry to state determinism and efficiency
A2	Remove Compiler	Directly interact with the model using natural language prompts	Verify the contribution of compiled communication to anti-contamination and efficiency
A3	Remove Cloud Memory	Retransmit the complete asset each time, without using cache	Verify the contribution of Cloud Memory to transmission efficiency and cost
A4	AgentOS full version	All components enabled	Baseline value

Expected results: AgentOS's token consumption remains stable and does not increase with the number of rounds; state recovery accuracy approaches 100%; anti-contamination decay is less than 5%; after ablating each component, performance degrades to varying degrees, validating the synergistic effect of the Trinity architecture.

10. Industrial Deployment Path

10.1 Tiered Deployment Path for Vendors

- **Phase 1 (0-6 months): Programmable Cache API.** Encapsulate and open an API based on existing KV cache capabilities, enabling cache persistence and billing by space, completing core technology validation.
- **Phase 2 (6-18 months): Work Order Protocol and Compiler SDK.** Release the protocol specification and SDK, launch an open-source runtime reference

implementation, establish a developer community, and build the protocol ecosystem.

- **Phase 3 (18-36 months): Fully Managed AgentOS Cloud Service.** Launch a complete solution covering local to cloud environments, deploy a multi-agent collaboration network and skill marketplace, and complete the ecosystem loop.

10.2 Strategic Value for Large Model Vendors

Being the first to deploy the AgentOS architecture will yield three distinct differentiation advantages:

- **Cost:** Significantly reduce redundant token transmission and computation, increase throughput per GPU, and build a cost moat in the price war.
- **Ecosystem:** Establish developer stickiness through Cloud Memory and the Work Order Protocol, escaping the homogeneous model parameter competition — the user migration cost shifts from "changing an API Key" to "abandoning half a year's accumulated cognitive assets."
- **Enterprise Market:** An architecture-level privacy solution precisely meets the data compliance requirements of government and enterprise clients, opening up the high-value ToB market.

11. Multi-Agent Collaboration and the Agent Internet Outlook

Extend the Work Order Protocol into a universal communication language for agents, supporting task delegation, result callbacks, and service discovery. When enough agents connect to a unified Work Order Protocol, a new Agent Internet will emerge. The Work Order Protocol is its "TCP/IP," the Agent service directory is its "DNS," and the skill marketplace is its "App Store." Agent capabilities from different domains and vendors can be freely combined like Lego bricks, creating powerful network effects.

12. Conclusion

We stand at a paradigm crossroads for Agent development. One path is to continue patching the inherent flaws of "context as memory" at the application layer with clever engineering, enduring linear efficiency losses and exponential complexity growth. The other path is the AgentOS path proposed in this paper — returning to the essence of computation and reconstructing from the system architecture level.

The AgentOS proposed in this paper, with its Trinity design of Registry, Compiler, and Cloud Memory, systematically addresses the core challenges facing current Agents in memory, communication, security, and business models. It is not a better memory

manager, but the first systematic practice of a more correct computing philosophy in the domain of intelligent agents.

Our goal is to transform the Agent from an "amnesia patient" who must read a long diary every time it wakes up to know who it is, into a "digital brain" with a structured self, efficient communication, and sovereign privacy. This is the infrastructure upon which general artificial intelligence can reliably operate.

References

- [1] Park J S, O'Brien K, Cai C J, et al. Generative Agents: Interactive Simulacra of Human Behavior[C]//Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. 2023: 1-22.
- [2] Packer C, Gao S, Wooders S, et al. MemGPT: Towards LLMs as Operating Systems[J]. arXiv preprint arXiv:2310.08560, 2023.
- [3] Lewis P, Perez E, Piktus A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J]. Advances in Neural Information Processing Systems, 2020, 33: 9459-9474.
- [4] Vaswani A, Shazeer N, Parmar N, et al. Attention is All You Need[J]. Advances in Neural Information Processing Systems, 2017, 30.
- [5] Ritchie D M, Thompson K. The UNIX Time-sharing System[J]. Communications of the ACM, 1974, 17(7): 365-375.
- [6] Solomon D A, Russinovich M E, Ionescu A. Windows Internals, Part 1[M]. Microsoft Press, 2012.
- [7] Tanenbaum A S, Bos H. Modern Operating Systems[M]. 4th ed. Pearson, 2014.
- [8] Shannon C E. A Mathematical Theory of Communication[J]. The Bell System Technical Journal, 1948, 27(3): 379-423.
- [9] Aho A V, Lam M S, Sethi R, et al. Compilers: Principles, Techniques, and Tools[M]. 2nd ed. Pearson Education, 2006.
- [10] Google Cloud. Context Caching Overview[EB/OL]. Vertex AI Documentation, 2024.
- [11] DeepSeek-AI. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model[J]. arXiv preprint arXiv:2405.04434, 2024.
- [12] Anthropic. Introducing Tool Use and Caching[EB/OL]. Anthropic Blog, 2024.
- [13] Regulation (EU) 2016/679 of the European Parliament and of the Council on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data (General Data Protection Regulation)[S]. 2016.
- [14] Saltzer J H, Schroeder M D. The Protection of Information in Computer Systems[J]. Proceedings of the IEEE, 1975, 63(9): 1278-1308.
- [15] Minsky M. A Framework for Representing Knowledge[R]. MIT-AI Laboratory Memo 306, 1974.
- [16] Cousot P, Cousot R. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints[C]//Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages. 1977: 238-252.

[17] Engelbart D C. Augmenting Human Intellect: A Conceptual Framework[R]. SRI Summary Report AFOSR-3223, 1962.

[18] Denis M, Lleras M. A Survey on the Memorization of Large Language Models[J]. arXiv preprint arXiv:2308.05544, 2023.

[19] Wei J, Wang X, Schuurmans D, et al. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models[J]. Advances in Neural Information Processing Systems, 2022, 35: 24824-24837.

[20] Bubeck S, Chandrasekaran V, Eldan R, et al. Sparks of Artificial General Intelligence: Early Experiments with GPT-4[J]. arXiv preprint arXiv:2303.12712, 2023.

[21] Liu X, Yu H, Zhang J, et al. AgentBench: Evaluating Large Language Models as Agents[C]//The Twelfth International Conference on Learning Representations. 2024.

[22] Wang G, Xie Y, Jiang Y, et al. Voyager: An Open-Ended Embodied Agent with Large Language Models[J]. arXiv preprint arXiv:2305.16291, 2023.

[23] FIPA. FIPA ACL Message Structure Specification[S]. Foundation for Intelligent Physical Agents, 2002.

[24] Xu Y, Zhuo H, Chen Q, et al. GraphRAG: Structured Retrieval-Augmented Generation for Knowledge-Intensive LLM Applications[J]. arXiv preprint arXiv:2404.16130, 2024.

Appendix

Appendix A: Complete Schema of the Standardized Work Order Protocol

See the complete definition in Section 5.2.

Appendix B: Standard Root Key Specification for the Registry

Root Key Name	Prefix	Core Purpose	Modification Permission
HKEY_LOCAL_AGENT	HKLA	Agent's own core configuration	Kernel mode, modifiable only during initialization
HKEY_CURRENT_USER	HKCU	Current user information and preferences	Writable in kernel mode, readable in user

			mode
HKEY_LOCAL_MACHINE	HKLM	Host environment configuration	System Administrator
HKEY_CURRENT_SESSION	HKCS	Current Session Runtime State	Kernel-Mode Writable
HKEY_CLOUD_SYNC	HKCSY	Cloud Sync State	Sync Service

Appendix C: Complete Definition of the System Call Interface

See the complete definition in Section 4.3.

Appendix D: Glossary

Terminology	English	Definition
AgentOS	Agent Operating System	Agent Operating System (AgentOS), the hierarchical architecture proposed in this paper
Registry	Registry	Tree-structured key-value store for deterministic metadata and state pointers
Compiler	Compiler	Intent purification and instruction translation component, bridging natural language and structured protocols
Work Order	Work Order	Structured inter-agent communication unit,

		serving as the interaction carrier between Agents and models/other Agents
Cloud Memory	Cloud Memory	Dedicated memory space co-located with the inference engine, hosting user cognitive assets
Four-Table Separation	Four-Table Separation	Memory model with logical separation of Registry, environment variables, system logs, and memory store
Three Separation Principles	Three Separation Principles	Separation of configuration and materials, separation of pointers and data, separation of companionship and execution
Cache Handshake Protocol	Cache Handshake Protocol	Standardized protocol for managing the lifecycle of cloud cache assets